Containers & Orchestration | Post-Class Summary

Module 5 · Topic 3 · Read this tonight. Look at it again before EPA.

The problem that started everything
Friday afternoon. Your app works perfectly on your laptop. You deploy to the client's server. Saturday morning - it's broken. Different Python version. Missing library. Wrong OS. This is the 'works on my machine' problem. Containers solve it by packaging the app AND its entire environment together in one sealed, portable unit.

CONTAINERS & DOCKER

Container

Think of it as: *A sealed lunchbox - everything the app needs is inside the lid. Open it anywhere, get the same result.*

What it does
Packages an app with its runtime, libraries, and config into one portable unit. Starts in milliseconds. Runs identically in dev, test, and production.

Why it matters at your org (Eg: Lloys, Ofgem, Ageas, Govt, Bylor)
Lloyds deploys containers for fraud detection, payments API, mobile banking. A traffic spike on payday? Hundreds of containers spin up in seconds. VMs would take minutes - and miss the rush.

EPA link: K14 - Explain what a container is and the problem it solves. K15 - Compare containers vs VMs for a given workload.

Docker Image vs Container

Think of it as: *Image = the master stamp (frozen template). Container = the printed copy (running instance).*

What it does
Image: read-only template built from a Dockerfile. Container: a live, running instance of that image. One image → stamp out 100 identical containers.

Why it matters at your org (Eg: Lloys, Ofgem, Ageas, Govt, Bylor)
One approved image in ECR. ECS stamps out however many containers are needed. Every copy is identical. No configuration drift. No surprises in production.

EPA link: K14 - Know the difference between image and container. S6 - You built the image (docker build) and ran the container (docker run) in Lab 8.1.

Dockerfile

Think of it as: *The recipe card - step-by-step instructions for packing the lunchbox.*

What it does
A text file that tells Docker exactly how to build the image: which base image to start from, which files to copy, which libraries to install, which command to run.

Why it matters at your org (Eg: Lloys, Ofgem, Ageas, Govt, Bylor)
In the lab, your Dockerfile defined the Node.js app environment precisely. Any engineer can build an identical image from it. That repeatability is non-negotiable in regulated financial services.

EPA link: S6 - You wrote a Dockerfile in Task 3 of Lab 8.1. K14 - Understand what a Dockerfile does and why it matters.

Virtual Machine
Full OS per app - gigabytes
Minutes to start
5–10 per server typically
3 apps = 3 OSs to patch

Container
Shared OS kernel - megabytes
Milliseconds to start
Hundreds per server
3 apps = 1 OS. One bill.

Amazon ECR - Lloyds' private fridge. Not Docker Hub (public - anyone can take). ECR is padlocked - IAM-controlled, encrypted at rest (S3 SSE), transfers over HTTPS. Every push and pull is logged in CloudTrail. In the lab you ran **docker push** to put your image here. In production at Lloyds, ECS pulls from it automatically.

Page 2 of 3 · Orchestration — Managing Containers at Scale

Automation
A single task runs automatically.
Example: back up the database every night at midnight.

Orchestration
MULTIPLE automated tasks coordinated into a workflow.

	Example: code commit → run tests → build image → push to ECR → deploy → rollback if it fails.

AWS ORCHESTRATION SERVICES

Amazon ECS — Elastic Container Service

Think of it as: *The airport operations manager — you declare what needs to run, ECS decides where and self-heals if anything fails.*

What it does
Manages containers across a cluster of EC2 instances. Places containers based on available CPU and RAM. Replaces failed containers automatically. Integrates natively with IAM, ELB, ECR.

Why it matters at your org (Eg: Lloys, Ofgem, Ageas, Govt, Bylor)
A new version of Lloyds' payments API is deployed. ECS spins up the new containers, routes traffic gradually to them (blue/green), confirms they're healthy, then removes the old ones. Zero downtime. No engineer touches a server.

EPA link: K14 -Explain what ECS does and why it's needed. K20 - Choose between ECS options for a given scenario.

ECS + EC2 - Own the flat
You manage the servers. Full control. You patch the OS, choose instance type, configure networking. Use when regulators need proof of exact hardware (FCA, ONR).

ECS + Fargate -Airbnb
AWS manages the servers invisibly. You declare CPU and RAM. No OS to patch. Smaller security surface. Faster to start. Ideal for customer-facing APIs with variable traffic.

KUBERNETES, DOCKER SWARM & MESOS

Tool	The analogy	Use when...
Kubernetes (K8s) *Industry standard*	*Global logistics network - warehouses (nodes), delivery vans (pods), packages (containers). Same toolset on-prem or in any cloud.*	Production systems at scale. When you need portability, rolling deployments, community tooling. AWS managed version = Amazon EKS. Used at Lloyds.
Docker Swarm *Simple, native*	*Small tight-knit office team - fast, direct, minimal setup. Built into Docker, same CLI commands you already know.*	Small to medium deployments. One developer, quick setup, already using Docker. Limited scale vs K8s.
Apache Mesos *Massive scale*	*City operations director - coordinates hospitals, schools, power plants all sharing the same city infrastructure. Multiple frameworks on one cluster.*	Heterogeneous workloads - containers alongside Spark jobs, legacy systems. Very large enterprises. Less common now K8s has matured.

Amazon EKS - Managed Kubernetes on AWS. AWS runs the control plane for you. Certified K8s - same tools, same community plugins, portable workloads. Lloyds uses EKS in production. EKS vs ECS: choose EKS for portability and K8s ecosystem; choose ECS for AWS-only, simpler setup. **K20: you must be able to argue which to choose for a given scenario.**

Why orchestration matters - the three that keep a CTO awake:

Security & Compliance - policy enforced automatically, every server, every deployment. No human error. FCA/ICO/ONR audit-ready always.

Reliability & Availability - self-healing: a container fails at 3am, it's replaced at 3am. Customers never notice. TSB didn't have this (£330m, CEO resigned).

Scalability - Storm Éowyn hits, payday traffic spikes, smart meters go live. Orchestration scales automatically. Pay for exactly what you use, nothing more.

CHEF, PUPPET & ANSIBLE - CONFIGURATION MANAGEMENT

The problem they solve: *200 servers need identical configuration. You could SSH into each one manually - 200 servers × 30 minutes = 100 hours of work, plus human error, plus doing it again every time something changes. These tools define the configuration as CODE and enforce it everywhere, automatically.*

Tool	Cooking analogy	Best for
Chef — Imperative *HOW*	*Chef gives you a numbered recipe: Step 1 do this, Step 2 do that. Ruby-based cookbooks. Tells the server exactly HOW to get configured.*	Developer-heavy platform teams. Reusable cookbook patterns across hundreds of identical servers. Bylor Engineering - auditability of every step.
Puppet — Declarative *WHAT*	*Puppet points at a photo of the finished dish: 'It must look like THIS.' Doesn't care how. Detects drift - if reality doesn't match the photo, Puppet corrects it automatically.*	Compliance and risk teams. FCA/ICO audit readiness. Ofgem - every server provably matches approved config at all times. Drift detection is the killer feature.
Ansible — Agentless *SIMPLE*	*Ansible sends a text message: here's a YAML file, run it on all servers via SSH. No agent installed. No Ruby to learn. Readable by your manager.*	DevOps quick automation. Ageas -patch 150 servers tonight? 20-line YAML playbook. Done. Lowest barrier to entry. Start here if you're new to automation.

THREE BEST PRACTICES YOU MUST KNOW FOR EPA

Idempotency
Run the script once - get the result. Run it ten times - same result. Not additive.
Like a thermostat: set to 20°C, room is 20°C. Set it again - still 20°C. Not 40°C.

Version Control everything
Ansible playbooks, Chef cookbooks, Puppet manifests - ALL in Git.
At Lloyds: config change causes outage → who changed what when → rollback in minutes. FCA loves an immutable audit trail.

Never hardcode secrets
No passwords, API keys, or connection strings directly in scripts. Use AWS Secrets Manager or HashiCorp Vault.
A playbook with hardcoded credentials on GitHub = security incident. At Lloyds, that means headlines.

YOUR KSBs — WHAT TODAY EVIDENCES

K14	**Container tools**	Docker, Kubernetes, ECS, EKS - the entire session. Lab 8.1 is your practical evidence. You can explain what a container is, build one, and deploy it.
K20	**Cloud service selection**	ECS vs EKS, Fargate vs EC2. You can argue which to choose based on a scenario - startup vs bank, speed vs compliance.
K17	**Automation & scripting**	Chef, Puppet, Ansible - you can explain the difference and choose the right tool. Idempotency, version control, secrets management.
K15	**Cloud architecture**	Containers vs VMs, orchestration patterns, self-healing infrastructure, ECR pipeline.
S6	**Deploy applications**	Lab 8.1 Tasks 3–6: Dockerfile → docker build → docker run → ECR push. You did this.

Before you sleep tonight or before end of the week - three things:

1. Watch the EKS cluster video linked on the consolidation slide - 15 minutes. You'll see K8s in the AWS console.
2. Write three sentences in your learning journal: What did I build today? What problem does it solve? How does it connect to my role?
3. Attempt Lab 8.2 - Running Containers on a Managed Service. The ECR image you built today is the starting point.

Evidence of reflection = evidence of learning. That goes straight into your EPA portfolio.

The judgment to choose the right tool for the right context - that's what EPA tests. And that can't be Googled.